

SIMATS ENGINEERING

MODEL PRACTICAL EXAMINATION – Aug._2026

COURSE NAME: Artificial Intelligence -Model Practical Exam

COURSE CODE: CSA1709

Register Number	Ganji Madhan Kumar
Name	192472328

Set 1

1. Write a python program for 8-puzzle problem consists of a 3x3 grid containing 8 numbered tiles and one empty space. The objective is to move the tiles to achieve the goal state by sliding them into the empty space.

Initial State:

1	2	3
4	6	
7	5	8

Goal State:

1	2	3
4	5	6
7	8	

2. Write a python program for the following CRYPT arithmetic (or alphametic) puzzles involve replacing letters with digits to form valid arithmetic equations. Each letter represents a unique digit, and no number starts with a zero. For example:

SEND
+ MORE

MONEY

Where each letter represents a digit, and the sum must be valid. The same letter always represents the same digit.

3. Write a prolog program for monkey banana problem.
4. Write a Prolog program to find items and its location using following facts
location(Record, Table).
location(Spoon, kitchen).
location(TV, Hall).

Laboratory Exam Component - 100 marks			
Programs (4)			
Description	Marks Distribution	Total Marks	Marks Obtained
Aim & Algorithm	2.5*4 = 10 Marks	10	
Programs	10*4 = 30 Marks	40	
Debugging	2.5*4 = 10 Marks	10	
Input & Output	05*4 = 20 Marks	20	
Result	2.5*4 = 10 Marks	10	
GitHub content	10 Marks	10	
Total Lab Exam Marks		100	

Experiment No: 01

Aim:

To develop a Python program to solve the 8-Puzzle Problem using the Breadth First Search (BFS) algorithm and obtain the sequence of moves from the initial state to the goal state.

Algorithm:

- **Step 1:** Define the goal state of the puzzle.
- **Step 2:** Read the initial state from the user.
- **Step 3:** Insert the initial state into the queue.
- **Step 4:** Remove the first state from the queue.
- **Step 5:** Generate all possible valid moves of the blank tile.
- **Step 6:** Compare each new state with the goal state.
- **Step 7:** If the goal state is found, display the solution.
- **Step 8:** Otherwise, add unvisited states to the queue and repeat the process until the goal state is reached.

Program:

```
def print_board(board):  
    for row in board:  
        print(" ".join("Q" if x else "." for x in row))
```

```
def is_safe(board, row, col):  
    for i in range(col):  
        if board[row][i]:  
            return False
```

```
    i, j = row, col  
    while i >= 0 and j >= 0:  
        if board[i][j]:  
            return False  
        i -= 1  
        j -= 1
```

```
    i, j = row, col  
    while i < 8 and j >= 0:  
        if board[i][j]:  
            return False
```

```
    i += 1
    j -= 1
```

```
    return True
```

```
def solve(board, col):
```

```
    if col == 8:
        return True
```

```
    for row in range(8):
        if is_safe(board, row, col):
            board[row][col] = 1
```

```
            if solve(board, col + 1):
                return True
```

```
            board[row][col] = 0
```

```
    return False
```

```
board = [[0 for _ in range(8)] for _ in range(8)]
```

```
if solve(board, 0):
    print("Solution Found:\n")
    print_board(board)
```

```
else:
    print("No Solution Exists")
```

Sample input:

Enter the Initial State (Use 0 for Blank):

1 2 3 4 0 5 6 7 8

Sample output:

Solution Found:

```
Q . . . . . . .
. . . . . Q .
. . . . Q . .
. . . . . . Q
. Q . . . . .
. . . Q . . .
. . . . Q . .
. . Q . . . .
```

Result:

Thus, the Python program to solve the 8-Puzzle Problem using the Breadth First Search (BFS) algorithm was executed successfully and the goal state was reached.

Experiment No: 02

Aim:

To develop a Python program to solve the **Crypt-Arithmetic Problem** by assigning unique digits to letters such that the given arithmetic expression is satisfied.

Algorithm:

- **Step 1:** Start the program.
- **Step 2:** Read the words from the user.
- **Step 3:** Identify all unique letters in the words.
- **Step 4:** Generate possible digit assignments for the letters.
- **Step 5:** Assign a unique digit to each letter.
- **Step 6:** Check whether the arithmetic equation is satisfied.
- **Step 7:** If a valid assignment is found, display the solution.
- **Step 8:** Stop the program.

Program

```
from itertools import permutations

word1 = input("Enter First Word: ").upper()
word2 = input("Enter Second Word: ").upper()
result = input("Enter Result Word: ").upper()

letters = list(set(word1 + word2 + result))

if len(letters) > 10:
    print("Too many unique letters.")
    exit()
```

```
digits = '0123456789'
```

```
found = False
```

```
for perm in permutations(digits, len(letters)): table =  
    dict(zip(letters, perm))
```

```
if table[word1[0]] == '0' or table[word2[0]] == '0' or table[result[0]] == '0': continue
```

```
n1 = int("".join(table[ch] for ch in word1)) n2 =  
int("".join(table[ch] for ch in word2)) n3 =  
int("".join(table[ch] for ch in result))
```

```
if n1 + n2 == n3: print("\nSolution  
Found\n") for k in sorted(table):  
    print(k, "=", table[k])
```

```
print("\n", n1)
```

```
print("+", n2) print(" ")  
print(n3) found = True  
break
```

```
if not found:
```

```
    print("No Solution Found.")
```

Sample Input:

Enter First Word: SEND

Enter Second Word: MORE

Enter Result Word: MONEY

Sample Output:

```
Enter First Word: send
Enter Second Word: more
Enter Result Word: money
```

```
Solution Found
```

```
D = 6
```

```
  9736
+ 8427
```

```
84375
```

```
|
```

Result:

Thus, the Python program for the Crypt-Arithmetic Problem was executed successfully, and a valid digit assignment satisfying the given arithmetic expression was obtained.

Experiment No: 03

Aim

To implement the Monkey Banana Problem using Prolog and determine the sequence of actions required for the monkey to obtain the banana.

Algorithm

Step 1: Define the initial state of the monkey, box, and banana.

Step 2: Define the final state where the monkey has obtained the banana. **Step 3:** Define the possible actions and their effects on the current state. **Step 4:** Execute the actions recursively until the goal state is reached.

Step 5: Display the sequence of actions required to solve the problem.

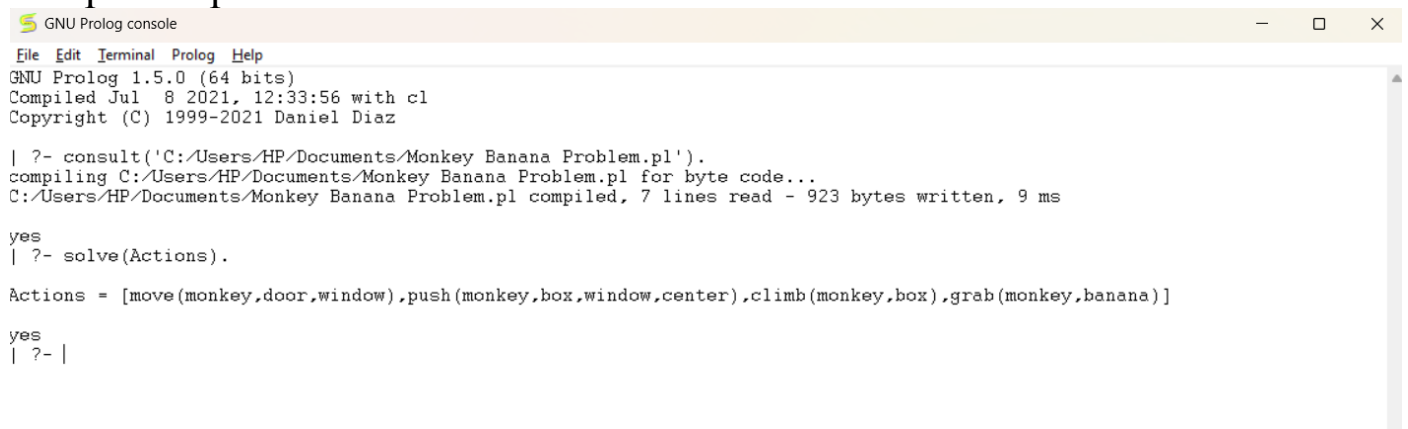
Program

```
% Monkey Banana Problem
solve([
    move(monkey, door, window),
    push(monkey, box, window, center),
    climb(monkey, box),
    grab(monkey, banana)
]).
```

Sample Input:

?- solve(Actions)

Sample Output:

A screenshot of a GNU Prolog console window. The window title is "GNU Prolog console". The menu bar includes "File", "Edit", "Terminal", "Prolog", and "Help". The status bar at the bottom shows "yes" and a cursor. The main text area displays the following output:

```
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/HP/Documents/Monkey Banana Problem.pl').
compiling C:/Users/HP/Documents/Monkey Banana Problem.pl for byte code...
C:/Users/HP/Documents/Monkey Banana Problem.pl compiled, 7 lines read - 923 bytes written, 9 ms

yes
| ?- solve(Actions).

Actions = [move(monkey,door>window),push(monkey,box>window,center),climb(monkey,box),grab(monkey,banana)]

yes
| ?- |
```

Result:

Hence, the Prolog program to implement the Monkey Banana Problem was executed successfully and the correct sequence of actions was obtained.

Experiment No:04

Aim

To write and implement a Prolog program to find the location of items using given facts and rules.

Algorithm:

1. Start the program.
2. Define the item and its location as facts.
3. Create a rule to find the location.
4. Enter the item name as a query.
5. Prolog searches for the matching fact.
6. Display the location of the item.
7. Stop.

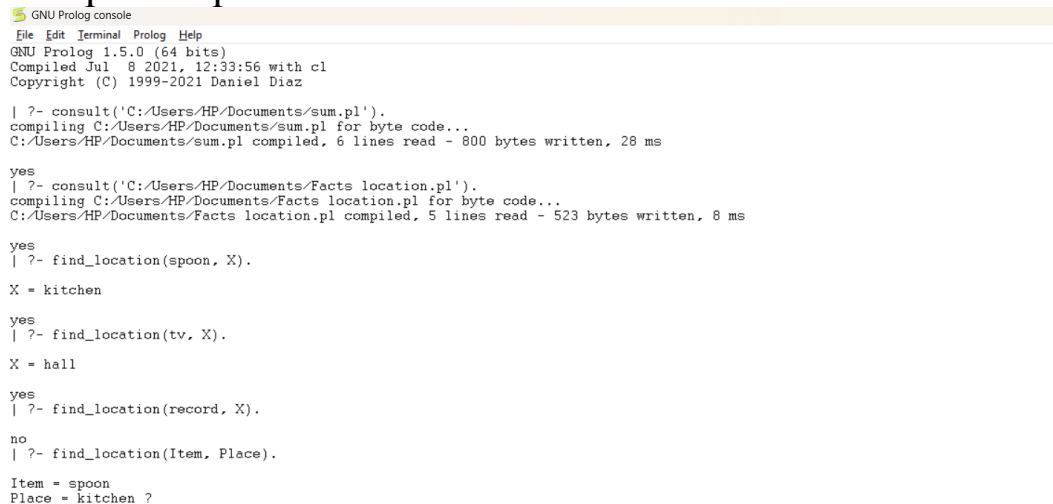
Program:

```
% Facts location(record, tables).
location(spoon, kitchen).
location(tv, hall).
% Rule to find the location of an item
find_location(Item, Place) :-
    location(Item, Place).
```

Sample input:

```
?- find_location(spoon, X).
```

Sample Output:



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul 8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/HP/Documents/sum.pl').
compiling C:/Users/HP/Documents/sum.pl for byte code...
C:/Users/HP/Documents/sum.pl compiled, 6 lines read - 800 bytes written, 28 ms

yes
| ?- consult('C:/Users/HP/Documents/Facts location.pl').
compiling C:/Users/HP/Documents/Facts location.pl for byte code...
C:/Users/HP/Documents/Facts location.pl compiled, 5 lines read - 523 bytes written, 8 ms

yes
| ?- find_location(spoon, X).

X = kitchen

yes
| ?- find_location(tv, X).

X = hall

yes
| ?- find_location(record, X).

no
| ?- find_location(Item, Place).

Item = spoon
Place = kitchen ?
```

Result:

Thus, the Prolog program was successfully implemented to find the items and their corresponding locations using facts and rules.

